

SECURITY AUDIT REPORT

Pocket Trainer Application

Document Control	Value
Assessment Type	Source-code and configuration security audit
Technology	Expo / React Native, Express, MongoDB Atlas / Mongoose, Modal AI
Repository	PocketTrainer
Assessment Date	1 September 2026
Classification	Confidential - Security Assessment
Method	Static review and safe dependency advisory checks

Prepared as an evidence-based defensive security assessment. No production, destructive, or external-service testing was performed.

Document Control

Field	Value
Report version	1.0
Scope	Repository snapshot available in D:\Projects\PocketTrainer
Source of truth	Application source, package manifests and lockfiles, configuration committed to repository
Out of scope	Cloud account settings, deployed environment variables, MongoDB Atlas controls, Modal configuration, Vercel project settings, mobile-device posture, external service tests
Assessment constraints	Static review; no exploit attempts, credential use, data mutation, or active scanning

Table of Contents

- 1. Executive Summary
- 2. Audit Scope and Methodology
- 3. Architecture and Threat Surface
- 4. Security Scorecard
- 5. Findings Summary
- 6. Detailed Security Findings
- 7. Dependency and Supply-Chain Assessment
- 8. Authentication and Authorization Assessment
- 9. API, Data Protection, Infrastructure, and Logging Assessments
- 10. Positive Controls, Risk Prioritization, and Roadmap
- 11. Manual Verification Items
- 12. Appendix

1. Executive Summary

Pocket Trainer is a mobile fitness application consisting of an Expo React Native client, a Node.js Express API deployed through Vercel configuration, MongoDB/Mongoose persistence, and a Modal-hosted AI service for workout generation and chat. The repository contains no Docker, Kubernetes, Terraform, CI/CD pipeline, test suite, or committed environment file.

Overall security posture: Needs Improvement. The implementation has a meaningful baseline of controls - bcrypt password hashing, protected API routes, hashed reset artifacts, HTTPS API configuration, Helmet, environment-variable references, and per-user query scoping - but it has significant account-security and AI-integrity gaps. The most urgent issue is a source-code JWT fallback secret that can enable token forgery if a deployment omits JWT_SECRET.

Severity	Count	Assessment
Critical	0	No unconditional critical issue established from repository evidence.
High	4	Authentication, credential storage, reset, and AI integrity concerns.
Medium	5	Input validation, error handling, concurrency, and dependency issues.
Low	1	Permissive browser CORS configuration.
Informational	3	Maturity and verification observations.

Top Five Risks

- JWT signing can fail open to a known fallback secret when JWT_SECRET is absent.
- Password-reset OTPs are predictable, logged in plaintext, and have no observed abuse controls.
- Bearer JWTs are stored in unencrypted React Native AsyncStorage.
- LLM-generated workout schedules are automatically written to MongoDB without server-side output validation.
- Authentication and costly AI endpoints have no rate limiting; the client dependency tree also has verified moderate npm advisories.

2. Audit Scope and Methodology

The audit inspected all repository-owned JavaScript/JSX source files, package manifests and lockfiles, Expo and EAS configuration, Vercel routing configuration, git ignore rules, and application architecture. Static review traced entry points, route protections, controller behavior, Mongoose queries, outgoing AI calls, local credential storage, and logging. npm audit --omit=dev was executed against both lockfiles using the npm advisory service.

Finding confidence reflects repository evidence, not assumptions about cloud settings. “Confirmed” means the condition is directly present in committed source. “Likely risk” means the behavior is present but exploitation depends on environmental conditions. “Needs Manual Verification” identifies security-relevant controls that cannot be validated from the repository.

Standards Reference

Relevant mappings use OWASP Top 10 2021, OWASP API Security Top 10 2023, CWE, and CVSS v3.1 where a reasonably bounded vector can be stated. Scores describe the code-level condition and may change with production deployment controls.

3. Architecture and Threat Surface

Boundary	Data / Control Flow	Primary Risk
Mobile client -> API	JWT bearer token; registration, login, reset, profile, workout, and chat requests	Token theft, brute force, malformed input, API abuse
API -> MongoDB	Mongoose queries and document writes for users, workout plans, and chat	Data integrity, tenant isolation, sensitive-data retention
API -> Modal AI	User biometrics, goals, prompts, plan context, and chat sent via axios	Prompt injection, data disclosure, untrusted structured output
API -> exercise media CDN	Hard-coded third-party GIF URLs rendered by client	Third-party content/privacy and availability
Deployment boundary	Vercel routing and secret-provided database/AI/JWT values	Secret configuration, network policy, observability

No file upload, file download, WebSocket, payment, administrator, role-management, Docker, Kubernetes, Terraform, reverse-proxy, or CI/CD implementation was found in the repository.

4. Security Scorecard

Domain	Rating	Reason
Authentication	Poor	Known JWT fallback, long-lived tokens, weak reset delivery controls, and no rate limiting.
Authorization	Needs Improvement	All observed private routes use middleware and tenant queries are generally self-scoped; user existence after JWT validation is not checked.
Input validation	Poor	No route-level validation despite Zod dependency; schemas have limited constraints.
API security	Needs Improvement	Helmet is present and API uses auth middleware, but CORS is unrestricted, error handling leaks internals, and abuse controls are absent.
Data protection	Needs Improvement	Sensitive biometric/chat data is stored and sent to AI; retention/encryption/cloud controls cannot be verified.
Cryptography	Needs Improvement	bcrypt and SHA-256 reset hashing are used; JWT fallback and Math.random OTP weaken design.
Secrets management	Needs Improvement	Environment variables are ignored correctly; startup does not enforce required secrets.
Dependencies	Needs Improvement	Backend audit clean; client audit has 11 moderate advisories.
Infrastructure/configuration	Needs Manual Verification	Vercel configuration is minimal; cloud network, TLS, IAM, and secret settings are not in repository.
Logging/monitoring	Poor	Reset OTP and provider error data may be logged; no structured security monitoring shown.
Business logic	Needs Improvement	Workout completion and AI plan mutation are non-atomic and insufficiently validated.
Overall	Needs Improvement	Baseline controls exist but high-risk account and AI controls require remediation before broad production use.

5. Findings Summary

ID	Severity	Title	Confidence
PT-01	High	JWT signing fails open to a known fallback secret	High
PT-02	High	Password-reset OTP workflow lacks cryptographic generation and abuse controls	High
PT-03	High	Bearer token stored in unencrypted mobile storage	High
PT-04	High	Unvalidated LLM output can automatically alter persisted workout plans	High
PT-05	Medium	Missing API schema validation allows malformed and oversized business inputs	High
PT-06	Medium	Internal error messages and external provider details can leak	High
PT-07	Medium	Workout state changes are non-atomic and replayable	High
PT-08	Medium	Password policy is not enforced server-side and sessions are not revoked	High
PT-09	Medium	Verified moderate advisories in Expo dependency tree	High
PT-10	Low	CORS accepts requests from every browser origin	High
PT-11	Info	No automated tests or CI security controls found	High
PT-12	Info	Sensitive-data governance and cloud hardening need manual verification	High
PT-13	Info	No confirmed SQL injection, command injection, upload, SSRF, XXE, or XSS sink found	Medium

6. Detailed Security Findings

PT-01 - JWT signing fails open to a known fallback secret

Attribute	Detail
Severity	High
Confidence	High
CVSS v3.1	8.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H; conditional on missing deployment secret)
CWE	CWE-798: Use of Hard-coded Credentials; CWE-321: Use of Hard-coded Cryptographic Key
OWASP	OWASP A02:2021 Cryptographic Failures; OWASP API2:2023 Broken Authentication
Affected component	Authentication / JWT
Affected file(s)	backend/controllers/authController.js:6-8; backend/middleware/authMiddleware.js:7-14

Description: The token generator signs JWTs with a fixed literal when JWT_SECRET is absent.

Technical analysis: Registration and login call generateToken, which selects process.env.JWT_SECRET or the committed literal. The middleware verifies with process.env.JWT_SECRET. In an environment where the secret is omitted, newly issued tokens use the known literal and an attacker can create valid tokens containing arbitrary id claims. The exact production secret configuration is not in the repository, so production exploitability requires manual verification.

Impact: Potential full account impersonation, unauthorized access to profiles, chat history, and workout data.

Attack scenario: An attacker recognizes a deployment with an unset JWT_SECRET, signs a token using the known fallback and a victim user ID, then calls protected endpoints.

Evidence: File: backend/controllers/authController.js, line 7: process.env.JWT_SECRET || [redacted fixed fallback].

Existing mitigations: jwt.verify rejects invalid signatures when a proper secret is configured.

Recommended remediation: Require JWT_SECRET during startup and terminate the process if it is missing or insufficiently random. Remove the fallback, restrict accepted algorithms, add iss/aud/sub claims, use short-lived access tokens, and implement refresh/session revocation.

Remediation priority: Immediate

PT-02 - Password-reset OTP workflow lacks cryptographic generation and abuse controls

Attribute	Detail
Severity	High
Confidence	High
CVSS v3.1	7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
CWE	CWE-330: Use of Insufficiently Random Values; CWE-532: Insertion of Sensitive Information into Log File; CWE-307: Improper Restriction of Excessive Authentication Attempts
OWASP	OWASP A07:2021 Identification and Authentication Failures; OWASP API2:2023 Broken Authentication
Affected component	Password reset
Affected file(s)	backend/controllers/authController.js:67-90; backend/routes/authRoutes.js:14-17

Description: Reset codes are generated with Math.random, logged with the user email, and protected by no observed request or verification limit.

Technical analysis: The code uses Math.random to select a six-digit OTP then hashes it for storage. Hashing and the 15-minute expiry are positive controls, but Math.random is not intended for secrets. The log statement is active in all environments despite a comment labeling it development-only. Routes have no rate limiter, attempt counter, cooldown, or delivery implementation.

Impact: Account takeover risk increases through OTP prediction/brute forcing or log access; the endpoint can also be abused for resource exhaustion.

Attack scenario: An attacker submits repeated reset and verification requests for a known account. A log reader can retrieve the plaintext OTP; repeated guesses can be attempted without application-level logout.

Evidence: File: backend/controllers/authController.js, lines 78-85; route registration at backend/routes/authRoutes.js, lines 14-17.

Existing mitigations: OTP digest is SHA-256 hashed before storage and expiry is 15 minutes; unknown emails receive a generic response.

Recommended remediation: Use crypto.randomInt(100000, 1000000), remove OTP logging, deliver OTPs through a trusted channel, rate-limit by IP and account, store attempt count, invalidate after a small number of failures, and monitor reset anomalies.

Remediation priority: Immediate

PT-03 - Bearer token stored in unencrypted mobile storage

Attribute	Detail
Severity	High
Confidence	High
CVSS v3.1	6.8 (CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)
CWE	CWE-922: Insecure Storage of Sensitive Information
OWASP	OWASP MASVS STORAGE; OWASP A02:2021 Cryptographic Failures
Affected component	Expo mobile client
Affected file(s)	api/axiosConfig.js:1-25; context/AuthContext.js:1-44

Description: Authentication JWTs are persisted in AsyncStorage rather than platform-backed encrypted storage.

Technical analysis: The client reads and writes userToken through @react-native-async-storage/async-storage and attaches it to every API call. The repository does not use expo-secure-store. AsyncStorage is not an encrypted credential vault.

Impact: A locally recovered JWT can be replayed until its 30-day expiry, exposing the account and user health-adjacent data.

Attack scenario: A compromised, rooted, debug-enabled, or physically accessible device exposes the stored token; the attacker replays it to the API.

Evidence: File: context/AuthContext.js, lines 13, 28, and 36; File: api/axiosConfig.js, lines 15-21.

Existing mitigations: HTTPS is used for the configured API URL, and logout removes the key.

Recommended remediation: Use expo-secure-store for refresh/session credentials; reduce access-token lifetime; rotate refresh tokens; invalidate sessions after credential changes; avoid logging bearer headers.

Remediation priority: Immediate

PT-04 - Unvalidated LLM output can automatically alter persisted workout plans

Attribute	Detail
Severity	High
Confidence	High
CVSS v3.1	7.1 (CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:U/C:N/I:H/A:L)
CWE	CWE-20: Improper Input Validation; CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes
OWASP	OWASP API8:2023 Security Misconfiguration; OWASP LLM01 Prompt Injection (contextual)
Affected component	AI service and plan mutation
Affected file(s)	backend/services/aiService.js:43-56, 118-138; backend/controllers/workoutController.js:15-37; backend/controllers/chatController.js:39-47

Description: AI output is parsed and used as application data without server-side schema validation; chat output can replace the full workout schedule automatically.

Technical analysis: Prompts request JSON but prompt compliance is not a security boundary. generateWorkoutPlan only JSON.parse values. In chat, an LLM response with action update_plan and updated_schedule replaces currentPlan.schedule. No Zod parse, field bounds, allowlist, plan-version check, confirmation, or transaction is present. User text and the full plan are interpolated into prompts, enabling instruction-conflict attempts.

Impact: Malformed, excessive, or manipulated workout data can be persisted, causing integrity loss and potentially safety-relevant exercise guidance. It is not evidence of server code execution.

Attack scenario: A user submits an adversarial chat message that causes the model to return update_plan with unsafe or malformed schedule content; the API saves it directly.

Evidence: File: backend/controllers/chatController.js, lines 43-47; File: backend/controllers/workoutController.js, lines 33-37; Zod is installed but unused.

Existing mitigations: Mongoose requires selected fields on workout-plan documents and prompts request a JSON shape.

Recommended remediation: Validate all model output with strict Zod schemas, reject unknown fields, bound lengths/counts/ranges, use allowlisted exercise data, require user confirmation or plan-versioning for mutation, and use a transaction/optimistic concurrency control.

Remediation priority: Immediate

PT-05 - Missing API schema validation allows malformed and oversized business inputs

Attribute	Detail
Severity	Medium
Confidence	High
CVSS v3.1	5.3 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:L)
CWE	CWE-20: Improper Input Validation; CWE-400: Uncontrolled Resource Consumption
OWASP	OWASP A03:2021 Injection; OWASP API8:2023 Security Misconfiguration
Affected component	Express API and Mongoose models
Affected file(s)	backend/server.js:15-18; backend/controllers/authController.js:12-177; backend/controllers/chatController.js:20-30; backend/models/*.js

Description: Request bodies are consumed directly without route schemas, field allowlists, length limits, or consistent server-side business validation.

Technical analysis: Express uses `express.json()` with default settings. Controllers destructure raw bodies, profile updates assign raw values, and message text is persisted after only non-empty checking. `User`, `ChatMessage`, `DailyLog`, and `WorkoutPlan` schemas lack practical max lengths, enums, numeric ranges, trims, and strict query policy. This is a confirmed validation weakness; no exploitable NoSQL injection sink was confirmed because observed find queries construct explicit object shapes.

Impact: Malformed data, validation-error leakage, oversized stored content, LLM cost amplification, and future injection/mass-assignment risk.

Attack scenario: An authenticated client submits very large chat text or invalid profile values; the API forwards/persists the input without bounded API schema validation.

Evidence: Files: `backend/server.js` lines 16-18; `backend/controllers/authController.js` lines 14-34 and 149-151; `backend/controllers/chatController.js` lines 22-30.

Existing mitigations: Mongoose required fields and casts provide partial validation; Zod is declared as a dependency.

Recommended remediation: Add strict Zod schemas at each route; reject unknown fields; enforce max request body/message sizes; constrain email, password, date, goal, numeric and plan fields; enable strict query/sanitize-filter protections as defense in depth.

Remediation priority: Short Term

PT-06 - Internal error messages and external provider details can leak

Attribute	Detail
Severity	Medium
Confidence	High
CVSS v3.1	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
CWE	CWE-209: Generation of Error Message Containing Sensitive Information; CWE-532: Insertion of Sensitive Information into Log File
OWASP	OWASP A09:2021 Security Logging and Monitoring Failures; OWASP API8:2023 Security Misconfiguration
Affected component	API error handling and logging
Affected file(s)	<code>backend/controllers/authController.js</code> :41-42, 62-63, 88-89, 122-123, 137-138, 155-156, 176-177; <code>backend/controllers/workoutController.js</code> :44-46; <code>backend/services/aiService.js</code> :53,136; <code>backend/config/db.js</code> :6-8

Description: Multiple controllers return `error.message` to clients and service code logs upstream provider payloads.

Technical analysis: Uncaught database, validation, or provider errors are surfaced through response JSON. AI service error logging includes `error.response.data` when present; it may include provider request context or diagnostic data. Reset OTP logging is addressed separately in PT-02.

Impact: Technology and data-flow disclosure can aid attacks; logs may contain sensitive prompt or provider diagnostic data.

Attack scenario: A malformed input triggers a Mongoose cast/validation error and the client receives internal details; a provider failure writes the provider response to logs.

Evidence: Files and lines above; e.g., `backend/controllers/workoutController.js`:46 returns `error.message`.

Existing mitigations: Chat controller uses a generic user-facing error message for its outer catch.

Recommended remediation: Use centralized error middleware, stable public error codes, correlation IDs, and structured redacted logs. Do not log provider payloads or sensitive prompts by default.

Remediation priority: Short Term

PT-07 - Workout state changes are non-atomic and replayable

Attribute	Detail
Severity	Medium
Confidence	High
CVSS v3.1	4.3 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N)
CWE	CWE-362: Race Condition; CWE-367: Time-of-check Time-of-use
OWASP	OWASP API6:2023 Unrestricted Access to Sensitive Business Flows
Affected component	Workout completion and plan assignment
Affected file(s)	backend/controllers/workoutController.js:33-41, 138-150

Description: Concurrent or repeated completion requests can advance `current_day_index` multiple times, and plan creation plus user assignment is not transactional.

Technical analysis: `completeWorkout` loads a user from authentication middleware, increments an in-memory field, then saves. It neither checks `isCompletedToday` nor performs an atomic conditional update. A retry/replay or simultaneous requests can race. Plan creation and setting `active_program_id` are separate writes, creating partial-state risk on failure.

Impact: Workout progress integrity can be corrupted; user experience and analytical data can become inaccurate.

Attack scenario: A client intentionally replays `POST /api/workouts/complete` concurrently, resulting in more than one day increment.

Evidence: File: `backend/controllers/workoutController.js`, lines 138-150; `isCompletedToday` is computed only in `getTodayWorkout` at lines 108-110.

Existing mitigations: Authentication is required and `active_program_id` is checked.

Recommended remediation: Use an atomic `findOneAndUpdate` with a same-day condition or transaction; create immutable `DailyLog` records with a unique user/day index; make completion idempotent; transact plan creation and assignment.

Remediation priority: Short Term

PT-08 - Password policy is not enforced server-side and sessions are not revoked

Attribute	Detail
Severity	Medium
Confidence	High
CVSS v3.1	5.0 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
CWE	CWE-521: Weak Password Requirements; CWE-613: Insufficient Session Expiration
OWASP	OWASP A07:2021 Identification and Authentication Failures
Affected component	Authentication lifecycle
Affected file(s)	<code>backend/controllers/authController.js</code> :14-34, 95-121, 162-175; <code>backend/services/auth</code> middleware

Description: The API accepts passwords with no evident server-side complexity/length policy and does not invalidate existing 30-day JWTs after password reset or change.

Technical analysis: A six-character check exists only in the reset screen client code; registration and reset endpoints do not enforce it server-side. JWTs only have expiration and no token version, session table, denylist, refresh rotation, or

revocation on password change/reset.

Impact: Weak credentials may be accepted and previously stolen bearer tokens remain valid after a password recovery event.

Attack scenario: An attacker with a stolen JWT continues accessing protected routes after the account owner changes their password.

Evidence: File: backend/controllers/authController.js lines 14-34, 95-121, 162-175; JWT expiry at line 7.

Existing mitigations: Passwords are bcrypt hashed with generated salts and password changes require the current password.

Recommended remediation: Enforce password policy and compromised-password checks server-side, use short-lived access tokens, and invalidate all sessions/token versions on reset, change, disablement, or anomaly.

Remediation priority: Short Term

PT-09 - Verified moderate advisories in Expo dependency tree

Attribute	Detail
Severity	Medium
Confidence	High
CVSS v3.1	Varies by advisory; npm audit reports moderate severity
CWE	CWE-787 and CWE-1285 reported for transitive uuid advisory
OWASP	OWASP A06:2021 Vulnerable and Outdated Components
Affected component	Mobile dependency supply chain
Affected file(s)	package.json:6-35; package-lock.json; npm audit output

Description: The root lockfile has 11 moderate vulnerability entries from the installed Expo dependency chain; the backend production tree reports zero known advisories.

Technical analysis: The completed npm audit found moderate entries affecting expo, expo-splash-screen, @expo/config, @expo/config-plugins, @expo/cli, related transitive components, and uuid via xcode. The audit recommends version changes that appear incompatible with the declared Expo SDK and should not be applied blindly.

Impact: Build tooling or development environment compromise surface may be increased; app runtime exploitability depends on the specific advisory and use path.

Attack scenario: A developer or build agent processes attacker-controlled project configuration through a vulnerable transitive tool.

Evidence: Tool evidence: npm.cmd audit --omit=dev, executed 1 September 2026. Root metadata: 11 moderate, 0 high, 0 critical. Backend metadata: 0 findings.

Existing mitigations: Lockfiles are committed; backend audit result is clean for production dependencies.

Recommended remediation: Use Expo-supported upgrade paths and expo-doctor compatibility checks; update to a supported patched SDK/version set, then retest builds. Add CI audit/SCA monitoring and review advisories individually.

Remediation priority: Short Term

PT-10 - CORS accepts requests from every browser origin

Attribute	Detail
Severity	Low
Confidence	High
CVSS v3.1	3.1 (CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N)
CWE	CWE-942: Permissive Cross-domain Policy
OWASP	OWASP API8:2023 Security Misconfiguration
Affected component	Express API
Affected file(s)	backend/server.js:15-18

Description: The API configures cors() without an origin allowlist.

Technical analysis: The default CORS middleware configuration permits any origin. Because authentication uses bearer headers rather than ambient cookies, this is not a confirmed CSRF compromise by itself; a malicious origin still needs access to a token. It expands browser attack surface and is inappropriate for a production API with known web clients.

Impact: Increased cross-origin API exposure and reduced containment if a token is exposed in a browser context.

Attack scenario: A malicious website makes cross-origin calls with a previously compromised bearer token from a browser environment.

Evidence: File: backend/server.js, line 17.

Existing mitigations: Helmet is enabled; the native client does not require browser CORS.

Recommended remediation: Restrict CORS to known production and development web origins, enumerate allowed methods/headers, and avoid credentials unless required.

Remediation priority: Medium Term

7. Dependency and Supply-Chain Assessment

Two separate npm projects are present. The backend package manifest contains Express, Mongoose, Helmet, bcryptjs, jsonwebtoken, Axios, dotenv, CORS, Zod, and @google/genai. The root manifest contains Expo SDK 57, React Native, Axios, AsyncStorage, and UI/navigation dependencies. Both lockfiles are present, which supports reproducible dependency resolution.

Project	Audit command	Result
backend	npm.cmd audit --omit=dev --json --prefix backend	0 known vulnerabilities; 153 production packages reported.
root mobile client	npm.cmd audit --omit=dev --json	11 moderate entries; 0 high; 0 critical; 627 total packages reported.

No CI/CD pipeline or automated dependency monitoring configuration was found. Caret ranges are broadly used in package manifests, while lockfiles constrain installed builds. The backend declares Zod but the application does not currently invoke it for validation.

8. Authentication and Authorization Assessment

All observed workout, chat, profile, and password-change routes use the protect middleware. The middleware extracts a Bearer token, verifies it, and loads a user by decoded id. Controllers primarily use req.user._id to scope chat and current-plan access. This is positive access-control design for the present endpoint set.

However, `protect` calls next even if `User.findById` returns null, so downstream code can assume a valid user object that may not exist. There are no roles, administrative endpoints, object-ID endpoints supplied by users, or cross-tenant plan lookup routes in the repository; no confirmed IDOR/BOLA was identified in the observed endpoints. The absence of a user-null check and session revocation should be remediated.

Evidence: `backend/middleware/authMiddleware.js:7-12`; `backend/routes/workoutRoutes.js:11-14`;
`backend/routes/chatRoutes.js:6-7`.

9. API, Data Protection, Infrastructure, and Logging Assessments

API Security

Helmet is applied globally. The API has no file upload/download, dynamic URL-fetching, shell execution, XML parser, template engine, WebSocket, or raw SQL use. Accordingly, no confirmed command injection, path traversal, SSRF, XXE, SQL injection, or unsafe file-upload issue was found. Mongoose queries are explicit rather than constructed from complete request bodies, reducing current NoSQL-injection evidence; missing validation remains a material weakness.

Data Protection and Cryptography

Passwords use `bcryptjs` with a generated salt, and reset codes are stored as SHA-256 digests with a 15-minute expiry. The application stores email, DOB, weight, goal, workout data, and chat data. MongoDB encryption-at-rest, transport security, retention, backups, access control, and data-location commitments cannot be verified from source. The Modal prompt includes age, weight, goal, active-plan context, and user chat content; this outbound disclosure should be subject to a privacy and vendor-data review.

Infrastructure and Configuration

`backend/vercel.json` routes all requests to `server.js`. `app.json` has no embedded API secret; the public EAS project ID is configuration metadata, not treated as a credential. `.gitignore` excludes `.env` and `backend/.env`, and no repository environment file or plaintext private key was found. Vercel runtime settings, TLS, WAF, function concurrency/timeouts, Atlas IP allowlists, database roles, secret rotation, and Modal endpoint authentication are Needs Manual Verification.

Logging and Monitoring

The repository uses console logging. It logs the MongoDB host, database connection errors, AI provider error data, plan update identifiers, and a plaintext reset code. No structured audit log, authentication event alerting, security-monitoring integration, retention policy, redaction layer, or incident response configuration was found.

10. Positive Security Controls, Risk Prioritization, and Roadmap

Positive Security Controls

- `bcryptjs` hashes passwords with generated salts rather than storing plaintext passwords (`backend/controllers/authController.js:25-26, 115-116, 171-172`).
- Reset codes are persisted as SHA-256 digests and expire after 15 minutes (`backend/controllers/authController.js:79-83`).
- Protected API routes consistently use authentication middleware (`backend/routes/authRoutes.js:19-21`; `workoutRoutes.js:11-14`; `chatRoutes.js:6-7`).
- Chat history is scoped by authenticated user ID, and workout plan access begins from the authenticated user `active_program_id`.
- Helmet is globally enabled (`backend/server.js:16`).
- The mobile API base URL uses HTTPS (`api/axiosConfig.js:5`).
- Environment files are ignored by Git; no actual credential was found in inspected tracked source.
- Mongoose schemas define required ownership fields for user-owned plan/chat/log objects.

Risk Prioritization

Priority	Risk addressed	Business outcome
P0	PT-01 through PT-04	Prevent account takeover, protect tokens, and stop unsafe AI-driven persistence.
P1	PT-05 through PT-09	Improve API robustness, session lifecycle, business integrity, and supply-chain exposure.
P2	PT-10 through PT-13	Reduce browser exposure and mature verification, monitoring, and governance.

Immediate Remediation

- Remove the JWT fallback and enforce required secrets at startup.
- Replace Math.random OTPs, remove OTP logging, and add reset/login rate limiting and attempt counters.
- Move persistent mobile credentials to expo-secure-store and shorten token lifetime.
- Apply strict server-side Zod schemas to requests and all LLM outputs before MongoDB mutation.
- Add per-user and per-IP AI quotas, timeouts, concurrency controls, and transactional plan mutation.

Short-Term Remediation

- Implement centralized error middleware and redacted structured logs.
- Make workout completion idempotent and atomic; create/uniquely index daily logs.
- Enforce password policy server-side and revoke sessions on reset/change.
- Upgrade the Expo dependency set through an Expo-supported compatibility path and add SCA/SAST in CI.
- Add Mongoose field bounds, strict query settings, indexes, and retention/delete workflows.

Medium- and Long-Term Remediation

- Restrict CORS, define outbound provider controls, and review third-party media privacy/integrity.
- Establish MongoDB Atlas least privilege, network restrictions, backup/restore testing, encryption, and audit logs.
- Create a threat model, privacy impact assessment, incident response playbook, and periodic access/secret review.
- Introduce automated integration/security tests for authorization, reset throttling, malformed inputs, AI-output validation, and concurrency.

Concise Implementation Pattern

```
Example policy pattern (illustrative, not applied): require JWT_SECRET at boot; schema.parse(req.body);
rateLimit({ keyGenerator: user/IP }); strictPlanSchema.parse(modelOutput); conditional atomic update;
redacted error handler.
```

11. Manual Verification Items

Item	Why manual verification is required
JWT_SECRET presence and rotation	Deployment secrets are not committed; confirms whether PT-01 is exploitable in production.
Vercel security settings	Confirm TLS, environment separation, WAF/rate limiting, logs, serverless limits, and access control.
MongoDB Atlas configuration	Confirm least-privilege users, IP allowlist/private networking, TLS, encryption, backup, auditing, and retention.
Modal endpoint	Confirm authentication, endpoint exposure, data retention/training policy, encryption, request quotas, and logging.
OTP delivery	No email/SMS delivery implementation appears in source; verify that reset code delivery is real, secure, and monitored.
Mobile build posture	Confirm release signing, jailbreak/root strategy, Android backup/debug settings, iOS keychain configuration, and expo-secure-store suitability.
Production CORS clients	Identify exact permitted browser origins before applying an allowlist.
Third-party GIF usage	Confirm contractual/privacy posture and whether remote content is acceptable for users.

12. Conclusion

Pocket Trainer has a viable baseline security foundation, particularly in password hashing, protected routes, environment-file exclusion, and HTTPS client configuration. It is not yet ready for a high-assurance production posture because account compromise controls, secure token persistence, untrusted AI output validation, input validation, error handling, and operational monitoring require material improvement. Remediating PT-01 through PT-04 before broader deployment is the highest-value risk reduction.

Appendix A - Audited Components

Backend: server entry point, database connection, middleware, all routes, controllers, models, AI/exercise services, package manifest/lockfile, Vercel configuration. Client: API configuration, AuthContext, screens, navigation, Expo/EAS/Metro configuration, package manifest/lockfile, git ignore rules. Non-source assets were inventoried but not subjected to malware analysis.

Appendix B - Tools and Evidence

- Static source/configuration review using repository file enumeration and targeted pattern searches.
- `npm.cmd audit --omit=dev --json` for root and backend lockfiles; executed 1 September 2026.
- No dynamic exploitation, data mutation, credential use, production scanning, or external endpoint probing was performed.
- Line references are based on the repository snapshot reviewed.

Appendix C - CWE and OWASP Mapping

Finding	Primary CWE	OWASP
PT-01	CWE-798, CWE-321	A02; API2
PT-02	CWE-330, CWE-532, CWE-307	A07; API2
PT-03	CWE-922	A02 / MASVS Storage
PT-04	CWE-20, CWE-915	API8 / LLM01 contextual
PT-05	CWE-20, CWE-400	A03; API8
PT-06	CWE-209, CWE-532	A09; API8
PT-07	CWE-362, CWE-367	API6
PT-08	CWE-521, CWE-613	A07
PT-09	CWE-787, CWE-1285	A06
PT-10	CWE-942	API8

Final Recommendations

The following actions are recommended to improve Pocket Trainer security posture, reduce account-compromise risk, and establish a repeatable security engineering baseline.

- 1 **1. Establish a 0-14 day security release gate:** Block broad production release until PT-01 through PT-04 are remediated, tested, and peer reviewed. This gate should require a non-empty JWT secret, secure token storage, reset abuse controls, and strict AI-output validation.
- 2 **2. Make identity controls fail closed:** Validate required deployment secrets at startup, remove fallback credentials, use 10-15 minute access tokens, rotate refresh tokens, and invalidate all sessions after password reset, password change, or suspicious activity.
- 3 **3. Replace the password-reset workflow:** Use `crypto.randomInt` for OTP generation, remove all OTP logging, deliver codes only through a trusted channel, and add IP/account rate limits, cooldowns, attempt counters, and monitoring.
- 4 **4. Treat AI output as untrusted input:** Implement strict Zod schemas for generated plans and chat actions, reject unknown fields, constrain repetitions/rest/exercise counts, and require user confirmation or versioned rollback before plan replacement.
- 5 **5. Add an API protection layer:** Apply route-specific validation, body-size limits, rate limits, an explicit CORS allowlist, and centralized error handling with correlation IDs and redacted logs.
- 6 **6. Protect biometric-adjacent data:** Document data inventory, retention, deletion, and vendor sharing. Minimize data sent to Modal, use least-privilege MongoDB access, and verify Atlas network/TLS/backup/audit settings.
- 7 **7. Build security into delivery:** Add CI checks for npm audit, SAST, secret scanning, dependency monitoring, and tests for authorization, OTP throttling, AI schema failures, and replay/concurrency behavior.
- 8 **8. Create an operating cadence:** Assign owners, due dates, and evidence requirements for each finding. Review authentication failures, reset activity, AI usage/cost, dependency advisories, and cloud access at least monthly.

Recommended success measure: all Immediate items verified in a staging environment before release, followed by quarterly evidence-based security reviews.